

Bloque 3 — Mini-especificaciones funcionales y técnicas

Banco y conciliación: B006 a B009

1. Propósito del bloque

Este bloque convierte los movimientos importados desde una cartola en registros operables dentro del sistema.

Incluye:

- Consulta y filtrado de movimientos.
- Visualización detallada del dato bancario original.
- Clasificación administrativa manual.
- Distribución de un movimiento entre uno o varios destinos financieros.
- Cálculo de monto aplicado.
- Cálculo de saldo pendiente.
- Actualización del estado de conciliación.
- Auditoría de clasificaciones y aplicaciones.

Al finalizar el bloque, el sistema podrá responder:

¿Qué movimientos existen?
¿Qué información original entregó el banco?
¿Cómo se clasificó el movimiento?
¿A qué operaciones del sistema fue aplicado?
¿Cuánto queda pendiente de conciliar?

2. Resumen del bloque

Código	Actividad	Horas	Recurso	Cobertura actual	Resultado
B006	Lista de movimientos	4 h	Frontend	Parcial	Consulta filtrable y paginada
B007	Detalle de movimiento	4 h	Frontend	Parcial	Vista integral y trazabilidad
B008	Clasificación manual	4 h	Backend	Requiere ampliación	Clasificación administrativa auditable

Código	Actividad	Horas	Recurso	Cobertura actual	Resultado
B009	Aplicaciones bancarias múltiples	10 h	Backend	Alta, con ajustes	Distribución transaccional del movimiento

El bloque suma **22 horas**.

Los criterios de la Carta Gantt son:

- B006: filtros por cuenta, fecha, monto, texto y estado.
- B007: datos originales, aplicaciones y trazabilidad.
- B008: tipo, contraparte y observación justificable.
- B009: un movimiento distribuido entre varios destinos financieros.

3. Secuencia funcional

```
B006 Lista de movimientos
→ B007 Detalle del movimiento
→ B008 Clasificación manual
→ B009 Aplicaciones bancarias múltiples
```

La dependencia es lógica:

1. Primero se localiza el movimiento.
2. Después se revisa su información original.
3. Luego se clasifica administrativamente.
4. Finalmente se distribuye entre operaciones del sistema.

No todos los movimientos deben clasificarse antes de ser aplicados cuando el destino sea evidente, pero la interfaz debe ofrecer ambas acciones desde el detalle.

4. Distinción entre dirección, clasificación y aplicación

4.1 Dirección bancaria

El campo actual:

```
MovimientoBancario.tipo
```

solo indica:

INGRESO
EGRESO

Este dato proviene de la cartola y no debe modificarse como parte de B008.

Ejemplos:

INGRESO: depósito de un cliente
INGRESO: anticipo recibido de factoring
EGRESO: entrega de fondo a trabajador
EGRESO: devolución a cliente

Dos movimientos pueden tener la misma dirección bancaria, pero distinta naturaleza administrativa.

4.2 Clasificación administrativa

La clasificación identifica la naturaleza probable del movimiento:

PAGO DE CLIENTE
ANTICIPO DE FACTORING
ENTREGA DE FONDO
DEVOLUCIÓN DE CLIENTE
REEMBOLSO DE RENDICIÓN
GASTO BANCARIO
OTRO

La clasificación:

- No crea una conciliación.
- No consume monto.
- No vincula por sí sola un movimiento con una factura o rendición.
- Puede servir como sugerencia para una aplicación posterior.
- Debe conservar quién la realizó y por qué.

4.3 Aplicación bancaria

La aplicación vincula una parte del movimiento con una operación real del sistema.

Ejemplo:

Movimiento bancario de ingreso: \$1.000.000 CLP

Aplicación 1:

Pago de cliente	\$700.000
Aplicación 2:	
Liquidación de factoring	\$300.000
Total aplicado:	\$1.000.000
Saldo por conciliar:	\$0
Estado: CONCILIADO	

El modelo actual calcula el monto aplicado sumando las aplicaciones activas y obtiene el saldo restando del monto del movimiento. También determina si queda no conciliado, parcialmente conciliado o conciliado.

5. Diferencia entre B009 y B010

B009 — Nivel bancario

Distribuye un movimiento entre operaciones financieras:

```
Movimiento bancario
├─ PagoCliente
├─ FlujoFactoring
├─ EntregaFondo
├─ PagoLiquidacionRendicion
└─ DevolucionPagoCliente
```

B010 — Nivel de facturas

Distribuye un pago de cliente entre una o varias facturas:

```
PagoCliente
├─ Factura A
├─ Factura B
└─ Factura C
```

Por tanto, el flujo puede ser:

```
Movimiento bancario
→ AplicacionMovimientoBancario
→ PagoCliente
→ AplicacionPagoFactura
→ FacturaVenta
```

B009 no debe implementar todavía la distribución entre facturas.

6. Cobertura actual del modelo

`MovimientoBancario` ya contiene:

- Cuenta bancaria.
- Importación de origen.
- Fechas.
- Ingreso o egreso.
- Monto.
- Saldo bancario.
- Descripción original.
- Referencias.
- Contraparte.
- Fingerprint.
- Estado de conciliación.
- Observaciones.

`AplicacionMovimientoBancario` ya permite vincular un movimiento con:

- Pago de cliente.
- Flujo de factoring.
- Entrega de fondo.
- Pago de liquidación de rendición.
- Devolución de pago.
- Otro destino.

El modelo también valida que:

- El monto sea positivo.
- Exista exactamente un destino.
- La suma aplicada no supere el movimiento.
- Los ingresos y egresos sean compatibles con algunos destinos.

No obstante, deben corregirse cuatro aspectos:

1. Falta una clasificación administrativa separada.
2. La validación `clean()` no se ejecuta automáticamente al llamar `save()`.
3. No se controla que las aplicaciones bancarias superen el monto del destino.
4. La dirección de `PagoLiquidacionRendicion` no se valida actualmente.

7. Modelo nuevo: clasificación del movimiento

Se propone incorporar:

```
class ClasificacionMovimientoBancario(ModeloAuditoria):  
    class Categoria(models.TextChoices):  
        PAGO_CLIENTE = "PAGO_CLIENTE", "Pago de cliente"
```

```

ANTICIPO_FACTORING = "ANTICIPO_FACTORING", "Anticipo de factoring"
LIQUIDACION_FACTORING = (
    "LIQUIDACION_FACTORING",
    "Liquidación de factoring",
)
ENTREGA_FONDO = "ENTREGA_FONDO", "Entrega de fondo"
DEVOLUCION_RESPONSABLE = (
    "DEVOLUCION_RESPONSABLE",
    "Devolución de responsable",
)
REEMBOLSO_RENDICION = (
    "REEMBOLSO_RENDICION",
    "Reembolso de rendición",
)
DEVOLUCION_CLIENTE = (
    "DEVOLUCION_CLIENTE",
    "Devolución a cliente",
)
COMISION_FACTORING = (
    "COMISION_FACTORING",
    "Comisión de factoring",
)
COBRO_RECURSO = "COBRO_RECURSO", "Cobro con recurso"
GASTO_BANCARIO = "GASTO_BANCARIO", "Gasto bancario"
TRANSFERENCIA_INTERNA = (
    "TRANSFERENCIA_INTERNA",
    "Transferencia entre cuentas propias",
)
OTRO_INGRESO = "OTRO_INGRESO", "Otro ingreso"
OTRO_EGRESO = "OTRO_EGRESO", "Otro egreso"
SIN_CLASIFICAR = "SIN_CLASIFICAR", "Sin clasificar"

class Origen(models.TextChoices):
    MANUAL = "MANUAL", "Manual"
    REGLA = "REGLA", "Regla automática"
    SUGERENCIA = "SUGERENCIA", "Sugerencia"

movimiento = models.ForeignKey(
    MovimientoBancario,
    on_delete=models.PROTECT,
    related_name="clasificaciones",
)
categoria = models.CharField(
    max_length=40,
    choices=Categoria.choices,
)
origen = models.CharField(
    max_length=20,
    choices=Origen.choices,
    default=Origen.MANUAL,
)

```

```

contraparte_normalizada = models.CharField(
    max_length=200,
    blank=True,
)
rut_contraparte_normalizado = models.CharField(
    max_length=20,
    blank=True,
)
observacion = models.TextField(blank=True)
activa = models.BooleanField(default=True)

```

7.1 Razón para un modelo separado

No se recomienda sobrescribir:

```

descripcion_original
contraparte
rut_contraparte
tipo

```

porque esos campos representan la información recibida desde el banco.

El nuevo modelo permite conservar:

```

Dato original
→ clasificación inicial
→ reclasificación posterior
→ regla automática futura
→ historial completo

```

Esto también prepara B012, donde se construirán reglas automáticas.

B006 — Lista de movimientos

1. Objetivo

Permitir localizar y revisar rápidamente los movimientos importados de todas las cuentas bancarias.

La vista debe responder preguntas como:

- ¿Qué ingresos entraron esta semana?
- ¿Qué movimientos siguen sin conciliar?
- ¿Qué movimientos pertenecen a una cuenta?
- ¿Qué movimiento contiene una determinada referencia?
- ¿Qué egresos superan un monto?

- ¿Qué movimientos están parcialmente aplicados?

2. Modelo principal

MovimientoBancario

Modelos relacionados:

Banco
CuentaBancaria
ImportacionCartola
ClasificacionMovimientoBancario
AplicacionMovimientoBancario

3. URL

/administracion/movimientos/

Nombre recomendado:

administracion:movimiento_list

4. Vista

```
class MovimientoBancarioListView(  
    LoginRequiredMixin,  
    PermissionRequiredMixin,  
    ListView,  
):  
    model = MovimientoBancario  
    permission_required = "administracion.view_movimientobancario"  
    paginate_by = 50
```

La consulta no debe implementarse completamente dentro de `get_queryset()`.

Se utilizará:

selectors/movimientos_bancarios.py

5. Filtros

Cuenta

- Banco.
- Cuenta bancaria.
- Moneda.
- Cuenta activa o inactiva.

Fecha

- Fecha desde.
- Fecha hasta.
- Tipo de fecha:
- Operación.
- Contable.
- Valor.

Por defecto se utilizará `fecha_operacion`.

Monto

- Monto mínimo.
- Monto máximo.

Dirección

- Ingreso.
- Egreso.

Estado

- No conciliado.
- Parcialmente conciliado.
- Conciliado.
- Excluido.

Clasificación

- Categoría administrativa.
- Sin clasificar.
- Clasificación manual.
- Clasificación automática futura.

Texto libre

La búsqueda deberá consultar:

- Descripción original.
- Referencia bancaria.
- Número de documento.

- Identificador externo.
- Contraparte.
- RUT de contraparte.
- Cuenta de contraparte.
- Observaciones.

Importación

- Archivo de cartola.
- Fecha de importación.
- Usuario que importó.

6. QuerySet y rendimiento

La consulta base deberá incluir:

```
MovimientoBancario.objects.select_related(
    "cuenta_bancaria",
    "cuenta_bancaria__banco",
    "importacion",
)
```

Para evitar consultas por fila, se anotará:

```
monto_aplicado_db
saldo_por_conciliar_db
cantidad_aplicaciones
```

Ejemplo conceptual:

```
.annotate(
    monto_aplicado_db=Coalesce(
        Sum(
            "aplicaciones__monto",
            filter=Q(aplicaciones__activa=True),
        ),
        Decimal("0.00"),
    ),
)
```

No se recomienda llamar a la propiedad `monto_aplicado` individualmente para cada fila del listado, porque generaría consultas adicionales.

7. Ordenamiento

Opciones:

- Fecha descendente.
- Fecha ascendente.
- Mayor monto.
- Menor monto.
- Estado.
- Cuenta bancaria.
- Contraparte.

Orden predeterminado:

```
fecha_operacion descendente  
id descendente
```

8. Columnas de la tabla

Columna	Contenido
Fecha	Fecha de operación
Cuenta	Banco y cuenta enmascarada
Tipo	Ingreso o egreso
Monto	Valor del movimiento
Descripción	Texto original resumido
Contraparte	Nombre o RUT
Clasificación	Categoría actual
Aplicado	Total aplicado
Pendiente	Saldo por conciliar
Estado	Badge
Acciones	Ver, clasificar, aplicar

Los ingresos y egresos deben distinguirse mediante texto e icono, no únicamente mediante color.

9. Totales del resultado filtrado

Sobre la tabla se mostrarán:

```
Cantidad de movimientos
Total de ingresos
Total de egresos
Monto aplicado
Saldo por conciliar
```

Estos totales deben calcularse sobre el filtro completo, no solo sobre la página visible.

Para conjuntos grandes se puede calcular un resumen separado mediante el selector.

10. Acciones rápidas

Cada fila podrá mostrar:

- Ver detalle.
- Clasificar.
- Aplicar.
- Ver importación de origen.

No se permitirá editar directamente los datos originales desde el listado.

11. Exportación

La exportación no forma parte obligatoria de B006; se desarrollará formalmente en B020.

En esta etapa se puede dejar preparado:

```
selector reutilizable
filtros persistentes
estructura de columnas
```

12. Template

```
templates/administracion/movimientos/
└─ list.html
```

Componentes:

```
_filtros.html
_tabla.html
_resumen.html
```

13. Permisos

`view_movimientobancario
clasificar_movimientobancario
aplicar_movimientobancario`

Los botones de clasificar y aplicar se mostrarán solo cuando corresponda.

14. Auditoría

La consulta normal de un listado no requiere registrar un evento por cada acceso.

Sí se deberá auditar:

- Exportaciones futuras.
 - Consulta de información sensible cuando la política lo requiera.
 - Cambios de clasificación.
 - Aplicaciones.
-

15. Pruebas

- Filtrar por cuenta.
 - Filtrar por rango de fechas.
 - Filtrar por ingreso y egreso.
 - Filtrar por estado.
 - Filtrar por monto mínimo y máximo.
 - Buscar por descripción.
 - Buscar por referencia.
 - Buscar por contraparte.
 - Mostrar movimientos sin clasificación.
 - Calcular monto aplicado.
 - Calcular saldo pendiente.
 - Paginar resultados.
 - Evitar consultas N+1.
 - Ocultar acciones sin permiso.
-

16. Criterios de aceptación

- El listado muestra los movimientos importados.
- Los filtros pueden combinarse.
- La paginación conserva los filtros.
- Los montos aplicados y pendientes son correctos.
- El listado no genera consultas por cada movimiento.
- El usuario puede abrir el detalle.

- La información original no se modifica desde la tabla.
 - Los permisos controlan las acciones disponibles.
-

17. Evaluación de esfuerzo

Las 4 horas permiten construir un listado funcional si ya existen:

- Plantilla base.
- Componente de filtros.
- Paginación.
- Permisos.
- Selectores transversales.

Los totales avanzados, optimización y pruebas podrían requerir horas adicionales si el volumen de movimientos es alto.

B007 — Detalle de movimiento

1. Objetivo

Mostrar toda la información relacionada con un movimiento bancario en una sola pantalla.

La Carta Gantt exige:

`Datos originales
Aplicaciones
Trazabilidad`

2. URL

`/administracion/movimientos/<pk>/`

Nombre:

`administracion:movimiento_detail`

3. Vista

```
class MovimientoBancarioDetailView(  
    LoginRequiredMixin,  
    PermissionRequiredMixin,  
    DetailView,  
):  
    model = MovimientoBancario  
    permission_required = "administracion.view_movimientobancario"
```

La consulta deberá precargar:

- Cuenta y banco.
- Importación.
- Clasificaciones.
- Aplicaciones activas e inactivas.
- Destinos de cada aplicación.
- Eventos de auditoría.

4. Secciones de pantalla

4.1 Encabezado

Mostrar:

- Fecha.
- Ingreso o egreso.
- Monto.
- Estado de conciliación.
- Cuenta bancaria.
- Clasificación actual.

Ejemplo:

```
Movimiento #1854  
  
INGRESO – $850.000 CLP  
Parcialmente conciliado  
Cuenta corriente principal ••••6789
```

4.2 Resumen financiero

Indicador	Valor
Monto del movimiento	\$850.000

Indicador	Valor
Monto aplicado	\$600.000
Saldo por conciliar	\$250.000
Aplicaciones activas	2

4.3 Datos originales del banco

- Fecha de operación.
- Fecha contable.
- Fecha valor.
- Descripción original completa.
- Referencia bancaria.
- Número de documento.
- Identificador externo.
- Contraparte.
- RUT de contraparte.
- Cuenta contraparte.
- Saldo bancario.
- Fingerprint.

El fingerprint podrá mostrarse parcialmente o dentro de una sección técnica.

Estos datos serán de solo lectura cuando el movimiento provenga de una importación.

4.4 Origen

- Archivo.
- Importación.
- Plantilla utilizada.
- Fecha de importación.
- Usuario.
- Número de fila original, cuando se incorpore este dato.

Se recomienda agregar a `MovimientoBancario`:

```
fila_origen = models.PositiveIntegerField(
    null=True,
    blank=True,
)
```

Esto facilita investigar errores de cartola.

4.5 Clasificación

Mostrar:

- Categoría actual.
- Origen de clasificación.
- Contraparte normalizada.
- Observación.
- Usuario.
- Fecha.

Acciones:

- Clasificar.
- Reclasificar.
- Ver historial.

4.6 Aplicaciones bancarias

Tabla:

Fecha	Destino	Referencia	Monto	Estado	Usuario
15-09-2026	Pago cliente	Pago #184	\$500.000	Activa	Usuario
15-09-2026	Factoring	Flujo #32	\$100.000	Activa	Usuario

Las aplicaciones inactivas aparecerán en una sección de historial, no mezcladas con las activas de forma confusa.

4.7 Trazabilidad

Línea de tiempo:

Importado

- clasificado
- aplicación creada
- aplicación modificada
- conciliación parcial
- conciliación completa

Cada evento indicará:

- Fecha.
- Usuario.
- Acción.
- Motivo.

- Valores anteriores y nuevos.
-

5. Acciones disponibles

Según estado y permisos:

```
[Clasificar]
[Agregar aplicación]
[Editar aplicaciones]
[Ver importación]
[Ver auditoría]
```

Posteriormente:

```
[Excluir]
[Reabrir]
[Reversar]
```

Estas últimas se implementarán en B015 y B016.

6. Restricciones visuales

Si el movimiento está conciliado:

- Se podrá consultar.
- No se modificarán aplicaciones sin una reversa controlada.
- El botón de nueva aplicación permanecerá oculto o deshabilitado.

Si está excluido:

- No podrá recibir aplicaciones.
 - Se mostrará el motivo de exclusión.
-

7. Template

```
templates/administracion/movimientos/
├── detail.html
├── _datos_originales.html
├── _clasificacion.html
├── _aplicaciones.html
└── _timeline.html
```

8. Pruebas

- Mostrar datos originales.
 - Mostrar importación.
 - Mostrar clasificación activa.
 - Mostrar historial de clasificaciones.
 - Mostrar aplicaciones activas.
 - Mostrar aplicaciones anuladas.
 - Calcular resumen.
 - Movimiento sin aplicaciones.
 - Movimiento conciliado.
 - Movimiento excluido.
 - Usuario sin permiso.
 - Precarga sin consultas N+1.
-

9. Criterios de aceptación

- La pantalla presenta todos los datos originales.
 - Se identifica claramente el archivo de origen.
 - Se visualiza el monto aplicado.
 - Se visualiza el saldo pendiente.
 - Las aplicaciones activas e inactivas se distinguen.
 - La clasificación tiene historial.
 - Las acciones respetan estado y permisos.
 - La trazabilidad permite reconstruir el proceso.
-

B008 — Clasificación manual

1. Objetivo

Permitir asignar una categoría administrativa a un movimiento sin alterar el dato bancario original.

La tarea debe registrar:

- Tipo o categoría.
 - Contraparte normalizada.
 - Observación justificable.
-

2. Modelo involucrado

Nuevo:

ClasificacionMovimientoBancario

Relacionado:

MovimientoBancario

3. Formulario

Campos:

- Categoría.
- Contraparte normalizada.
- RUT normalizado.
- Observación.
- Confirmación.

Datos de contexto de solo lectura:

- Fecha.
- Ingreso o egreso.
- Monto.
- Descripción original.
- Contraparte original.
- Referencia.

4. Categorías compatibles por dirección

Para ingresos

PAGO_CLIENTE
ANTICIPO_FACTORING
LIQUIDACION_FACTORING
DEVOLUCION_RESPONSABLE
TRANSFERENCIA_INTERNA
OTRO_INGRESO

Para egresos

ENTREGA_FONDO
REEMBOLSO_RENDICION
DEVOLUCION_CLIENTE
COMISION_FACTORING
COBRO_RECURSO
GASTO_BANCARIO
TRANSFERENCIA_INTERNA
OTRO_EGRESO

El formulario debe limitar las opciones según `MovimientoBancario.tipo`.

5. Reglas de negocio

1. No se modifica el tipo bancario original.
 2. Solo puede existir una clasificación activa.
 3. Reclasificar desactiva la anterior.
 4. La clasificación anterior no se elimina.
 5. Las categorías "OTRO" requieren observación.
 6. La contraparte normalizada no reemplaza la contraparte original.
 7. Un movimiento conciliado solo podrá reclasificarse con permiso especial.
 8. Un movimiento excluido podrá clasificarse, pero no aplicarse.
 9. La clasificación no cambia automáticamente el estado de conciliación.
 10. La clasificación no crea aplicaciones.
-

6. Servicio

```
services/clasificacion_movimientos.py
```

Función:

```
@transaction.atomic
def clasificar_movimiento(
    *,
    movimiento_id,
    categoria,
    contraparte_normalizada,
    rut_contraparte_normalizado,
    observacion,
    usuario,
):
    movimiento = (
        MovimientoBancario.objects
        .select_for_update()
        .get(pk=movimiento_id)
    )

    # Validar dirección y categoría.
    # Desactivar clasificación anterior.
    # Crear nueva clasificación.
    # Registrar auditoría.
```

7. URLs

```
/administracion/movimientos/<pk>/clasificar/  
/administracion/movimientos/<pk>/clasificaciones/
```

8. Vistas

```
MovimientoClasificarView  
MovimientoHistorialClasificacionesView
```

La clasificación debe realizarse mediante `POST`.

9. Template

Se recomienda un modal o una página breve:

```
templates/administracion/movimientos/  
├─ clasificar.html  
└─ historial_clasificaciones.html
```

10. Sugerencia de aplicación

Después de clasificar, la interfaz puede mostrar:

```
Clasificación guardada: PAGO DE CLIENTE  
  
[Crear aplicación a pago de cliente]
```

Esto es solo un acceso rápido. La aplicación requiere una confirmación independiente.

11. Permisos

```
clasificar_movimientobancario  
reclasificar_movimientobancario  
ver_historial_clasificacion
```

12. Auditoría

Registrar:

- Clasificación anterior.
- Nueva clasificación.
- Contraparte normalizada.
- Usuario.
- Fecha.
- Observación.
- Origen manual.

La observación será obligatoria al reclasificar un movimiento conciliado.

13. Pruebas

- Clasificar ingreso con categoría de ingreso.
 - Rechazar categoría de egreso en un ingreso.
 - Clasificar egreso correctamente.
 - Exigir observación para "OTRO".
 - Reclasificar.
 - Desactivar clasificación anterior.
 - Conservar historial.
 - Impedir reclasificación sin permiso.
 - No modificar datos originales.
 - No cambiar estado de conciliación.
 - Registrar auditoría.
-

14. Criterios de aceptación

- El movimiento puede clasificarse.
 - Las opciones son compatibles con su dirección.
 - Se conserva la información bancaria original.
 - Solo existe una clasificación activa.
 - Las reclasificaciones dejan historial.
 - La clasificación no consume monto.
 - La clasificación no equivale a conciliación.
 - Las acciones quedan auditadas.
-

B009 — Aplicaciones bancarias múltiples

1. Objetivo

Distribuir el monto de un movimiento bancario entre una o varias operaciones financieras del sistema.

Ejemplos:

Un depósito corresponde a:

- dos pagos registrados;
- un anticipo de factoring;
- una devolución del responsable.

O:

Un egreso agrupado corresponde a:

- una entrega de fondo;
- un reembolso;
- una devolución a cliente.

2. Modelo principal

AplicacionMovimientoBancario

El modelo actual funciona como puente transversal y admite cinco destinos estructurados, además de OTRO .

3. Destinos disponibles

Destino	Modelo	Dirección esperada
Pago de cliente	PagoCliente	Ingreso
Flujo de factoring	FlujoFactoring	Según dirección del flujo
Entrega de fondo	EntregaFondo	Egreso
Pago de liquidación	PagoLiquidacionRendicion	Depende del tipo
Devolución de pago	DevolucionPagoCliente	Egreso
Otro	Sin FK	Según justificación

4. Corrección requerida: pago de liquidación

PagoLiquidacionRendicion puede representar:

DEVOLUCION_RESPONSABLE
REEMBOLSO_EMPRESA

La devolución del responsable genera un ingreso.

El reembolso de la empresa genera un egreso.

La validación debe incorporar:

```
if destino == Destino.PAGO_LIQUIDACION:  
    if pago_liquidacion.tipo == DEVOLUCION_RESPONSABLE:  
        tipo_esperado = MovimientoBancario.Tipo.INGRESO  
    else:  
        tipo_esperado = MovimientoBancario.Tipo.EGRESO
```

Actualmente esta dirección no está contemplada en `clean()`.

5. Corrección requerida: límite del destino

La validación actual controla que las aplicaciones no superen el movimiento, pero también debe comprobar que no superen la operación de destino.

Ejemplo incorrecto:

```
PagoCliente registrado: $500.000  
Aplicaciones bancarias:  
- Movimiento A: $400.000  
- Movimiento B: $300.000  
  
Total vinculado: $700.000
```

El sistema debe impedirlo.

Para cada destino:

```
Total de aplicaciones bancarias activas  
≤ monto de la operación de destino
```

Esto aplica a:

- PagoCliente.
- FlujoFactoring.
- EntregaFondo.
- PagoLiquidacionRendicion.

- DevolucionPagoCliente.

6. Aplicación parcial a un destino

Debe permitirse que una operación de destino sea cubierta por más de un movimiento.

Ejemplo:

```
Pago de cliente: $1.000.000

Movimiento A aplicado: $600.000
Movimiento B aplicado: $400.000
```

Por tanto, no se impondrá una relación uno a uno.

Se controlará únicamente que el total no supere el destino.

7. Restricción de integridad

La validación actual vive principalmente en `clean()`, pero Django no ejecuta automáticamente `full_clean()` dentro de `save()`.

Por eso se requiere:

1. Que todas las aplicaciones se creen mediante el servicio.
2. Que el servicio llame `full_clean()`.
3. Que se agregue una restricción de base para la combinación válida de destino y FK.

Ejemplo conceptual:

```
CheckConstraint(
    condition=(
        Q(
            destino="PAGO_CLIENTE",
            pago_cliente__isnull=False,
            flujo_factoring__isnull=True,
            entrega_fondo__isnull=True,
            pago_liquidacion__isnull=True,
            devolucion_pago__isnull=True,
        )
        |
        Q(
            destino="FLUJO_FACTORING",
            pago_cliente__isnull=True,
            flujo_factoring__isnull=False,
            ...
```

```

    )
    |
    Q(
        destino="OTRO",
        pago_cliente__isnull=True,
        flujo_factoring__isnull=True,
        entrega_fondo__isnull=True,
        pago_liquidacion__isnull=True,
        devolucion_pago__isnull=True,
    )
),
name="chk_adm_aplicacion_destino_valido",
)

```

8. Servicio transaccional

services/aplicaciones_bancarias.py

Función principal:

```

@transaction.atomic
def reemplazar_aplicaciones_movimiento(
    *,
    movimiento_id,
    aplicaciones,
    usuario,
    motivo="",
):
    movimiento = (
        MovimientoBancario.objects
        .select_for_update()
        .get(pk=movimiento_id)
    )

    # Verificar estado.
    # Bloquear destinos involucrados.
    # Validar todas las aplicaciones.
    # Verificar total del movimiento.
    # Verificar total de cada destino.
    # Desactivar aplicaciones reemplazadas.
    # Crear nuevas aplicaciones.
    # Actualizar estado de conciliación.
    # Registrar auditoría.

```

9. Política de modificación

No se recomienda editar silenciosamente una aplicación financiera existente.

La operación será:

```
Aplicación anterior
→ se marca inactiva
→ se registra motivo
→ se crea nueva aplicación
```

Esto conserva la trazabilidad.

Para ello conviene agregar:

```
anulada_en = models.DateTimeField(null=True, blank=True)
anulada_por = models.ForeignKey(...)
motivo_anulacion = models.TextField(blank=True)
```

Alternativamente, la información puede residir en `EventoAuditoria`, pero los campos directos facilitan consultas operativas.

10. Estado del movimiento

Después de cada cambio:

```
movimiento.actualizar_estado_conciliacion()
```

Resultados:

Total aplicado	Estado
0	NO_CONCILIADO
Menor que movimiento	PARCIAL
Igual al movimiento	CONCILIADO
Excluido manualmente	EXCLUIDO

La lógica existente ya contempla estos estados.

11. Reglas de negocio

1. El monto debe ser mayor que cero.

2. La suma activa no puede superar el movimiento.
3. La suma activa puede ser menor que el movimiento.
4. Exactamente un destino debe informarse por aplicación.
5. El destino debe coincidir con su FK.
6. La dirección bancaria debe ser compatible.
7. El destino no debe estar anulado.
8. La suma aplicada al destino no puede superar su monto.
9. Un movimiento excluido no puede recibir aplicaciones.
10. Un movimiento conciliado no se modifica sin reversa.
11. Las aplicaciones no se eliminan físicamente.
12. Toda modificación requiere usuario.
13. Las modificaciones de movimientos conciliados requieren motivo.
14. No se aceptan montos con más decimales que la moneda.
15. El saldo final no puede ser negativo.

12. Interfaz de aplicaciones

Desde el detalle se abrirá:

```
/administracion/movimientos/<pk>/aplicaciones/
```

Encabezado:

```
Movimiento de ingreso
Monto: $1.000.000 CLP
Aplicado: $600.000
Disponible: $400.000
```

Tabla editable:

Destino	Operación	Monto	Glosa	Acción
Pago cliente	Pago #184	\$500.000	Transferencia cliente	Quitar
Factoring	Flujo #32	\$100.000	Liquidación	Quitar
—	—	—	—	Agregar

13. Selección dinámica del destino

Al escoger:

```
PAGO_CLIENTE
```

el segundo selector debe mostrar pagos:

- No anulados.
- Con saldo bancario pendiente.
- Compatibles con el movimiento.
- Ordenados por fecha y monto.

Para:

ENTREGA_FONDO

mostrar:

- Entregas no anuladas.
- Aún no completamente vinculadas.
- Con forma de entrega compatible.

Para:

FLUJO_FACTORING

mostrar:

- Flujos con dirección compatible.
- Con saldo pendiente de vinculación.

14. Selector de operaciones

Se recomienda un endpoint interno para búsqueda:

/administracion/movimientos/<pk>/destinos/?tipo=PAGO_CLIENTE&q=...

Respuesta:

```
{
  "results": [
    {
      "id": 184,
      "text": "Pago #184 – Cliente X – $500.000",
      "monto": "500000.00",
      "aplicado": "0.00",
      "disponible": "500000.00"
    }
  ]
}
```

```
]
}
```

Este endpoint debe exigir autenticación y permisos.

15. Uso de formsets

Puede utilizarse un `formset` o una estructura equivalente.

Cada fila tendrá:

- Destino.
- Identificador del objeto.
- Monto.
- Glosa.
- Marcar para quitar.

El backend no debe confiar en los totales calculados por JavaScript.

16. JavaScript

Funciones visuales:

- Agregar fila.
- Eliminar fila.
- Filtrar destinos.
- Mostrar saldo disponible.
- Recalcular suma.
- Alertar si se supera el movimiento.

La validación definitiva permanece en Python.

17. URLs

```
/administracion/movimientos/<pk>/aplicaciones/  
/administracion/movimientos/<pk>/aplicaciones/agregar/  
/administracion/movimientos/<pk>/aplicaciones/<aplicacion_id>/anular/  
/administracion/movimientos/<pk>/destinos/
```

18. Vistas

```
MovimientoAplicacionesView  
MovimientoAplicacionCreateView
```

```
MovimientoAplicacionAnularView  
MovimientoDestinosSearchView
```

La creación múltiple puede gestionarse en una sola vista de servicio.

19. Permisos

```
view_aplicacionmovimientobancario  
add_aplicacionmovimientobancario  
change_aplicacionmovimientobancario  
anular_aplicacionmovimientobancario
```

Para modificar movimientos ya conciliados:

```
reabrir_movimientobancario
```

20. Auditoría

Cada operación deberá registrar:

- Movimiento.
- Estado anterior.
- Estado nuevo.
- Aplicaciones anteriores.
- Aplicaciones nuevas.
- Montos.
- Destinos.
- Usuario.
- Fecha.
- Motivo.

Ejemplo:

```
{  
  "estado_conciliacion": {  
    "anterior": "PARCIAL",  
    "nuevo": "CONCILIADO"  
  },  
  "aplicaciones_agregadas": [  
    {  
      "destino": "PAGO_CLIENTE",  
      "objeto_id": 184,  
      "monto": "400000.00"  
    }  
  ]  
}
```

```
]
}
```

21. Pruebas

Movimiento

- Aplicar parcialmente.
- Aplicar completamente.
- Rechazar exceso.
- Rechazar monto cero.
- Rechazar monto negativo.
- Impedir aplicar movimiento excluido.
- Impedir modificar conciliado sin permiso.

Destinos

- Pago de cliente con ingreso.
- Rechazar pago de cliente con egreso.
- Entrega de fondo con egreso.
- Rechazar entrega con ingreso.
- Flujo de factoring de ingreso.
- Flujo de factoring de egreso.
- Devolución de responsable con ingreso.
- Reembolso de empresa con egreso.
- Devolución a cliente con egreso.

Límites

- No superar movimiento.
- No superar destino.
- Permitir destino parcialmente cubierto por varios movimientos.
- Impedir doble aplicación al mismo destino cuando excede.
- Confirmaciones concurrentes.

Integridad

- Exactamente una FK.
- FK compatible con `destino`.
- `OTRO` sin FK.
- `full_clean()` ejecutado.
- Rollback completo ante error.
- Auditoría correcta.

22. Criterios de aceptación

- Un movimiento puede tener varias aplicaciones.
- Una operación puede recibir aplicaciones desde varios movimientos.

- El total aplicado no supera el movimiento.
- El total aplicado no supera el destino.
- Las direcciones son compatibles.
- El estado se recalcula automáticamente.
- Las aplicaciones inactivas permanecen en el historial.
- Las modificaciones son transaccionales.
- No existen aplicaciones con destino incoherente.
- Los conflictos concurrentes no generan sobreaplicaciones.
- Toda modificación queda auditada.

23. Evaluación de esfuerzo

Las 10 horas de B009 permiten implementar un MVP funcional apoyándose en:

- Modelos ya existentes.
- Permisos transversales.
- Auditoría transversal.
- Componentes de formularios.
- Vistas base.

La validación concurrente, restricciones de base, búsqueda dinámica y pruebas exhaustivas pueden requerir tiempo adicional. B009 es una de las tareas de mayor riesgo financiero del proyecto y no debería cerrarse solo porque la pantalla funciona.

8. Estructura de archivos del bloque

```
administracion/
├── models/
│   └── bancos.py
├── forms/
│   ├── movimientos.py
│   ├── clasificacion_movimientos.py
│   └── aplicaciones_bancarias.py
├── views/
│   ├── movimientos.py
│   ├── clasificacion_movimientos.py
│   └── aplicaciones_bancarias.py
├── services/
│   ├── clasificacion_movimientos.py
│   └── aplicaciones_bancarias.py
└── selectors/
    └── movimientos_bancarios.py
```

```

├── destinos_bancarios.py
├── templates/administracion/movimientos/
│   ├── list.html
│   ├── detail.html
│   ├── clasificar.html
│   ├── historial_clasificaciones.html
│   ├── aplicaciones.html
│   ├── _filtros.html
│   ├── _tabla.html
│   ├── _resumen.html
│   ├── _datos_originales.html
│   ├── _clasificacion.html
│   ├── _aplicaciones.html
│   └── _timeline.html
├── static/administracion/js/
│   └── aplicaciones_bancarias.js
└── tests/
    ├── test_movimiento_list.py
    ├── test_movimiento_detail.py
    ├── test_clasificacion_movimiento.py
    ├── test_aplicaciones_bancarias.py
    └── test_aplicaciones_concurrencia.py

```

9. Migraciones previstas

Migración 1 — Clasificación

- Crear `ClasificacionMovimientoBancario`.
- Agregar índices por movimiento, categoría y estado activo.

Migración 2 — Trazabilidad de aplicación

Agregar, según decisión final:

```

anulada_en
anulada_por
motivo_anulacion

```

Migración 3 — Integridad

- Agregar restricción entre `destino` y FK.
- Revisar índices de aplicaciones.

- Agregar índice por `movimiento` y `activa`.

Migración 4 — Origen de fila

Agregar opcionalmente:

```
MovimientoBancario.fila_origen
```

10. Índices recomendados

En `MovimientoBancario`:

```
models.Index(  
    fields=["cuenta_bancaria", "fecha_operacion"],  
)  
models.Index(  
    fields=["cuenta_bancaria", "estado_conciliacion"],  
)  
models.Index(  
    fields=["tipo", "fecha_operacion"],  
)
```

En `AplicacionMovimientoBancario`:

```
models.Index(  
    fields=["movimiento", "activa"],  
)  
models.Index(  
    fields=["destino", "activa"],  
)
```

En clasificación:

```
models.Index(  
    fields=["movimiento", "activa"],  
)  
models.Index(  
    fields=["categoria", "activa"],  
)
```

11. Definición de terminado del bloque

B006–B009 se considerará terminado cuando:

- Los movimientos pueden listarse.
- Los filtros son combinables.
- El listado está paginado.
- No existen consultas N+1.
- El detalle muestra datos originales.
- El detalle muestra importación y trazabilidad.
- Los movimientos pueden clasificarse.
- La clasificación no altera datos bancarios.
- Las reclasificaciones conservan historial.
- Un movimiento puede distribuirse entre varios destinos.
- Una operación puede vincularse parcialmente con varios movimientos.
- No se supera el monto del movimiento.
- No se supera el monto de la operación destino.
- Las direcciones son compatibles.
- El estado de conciliación se recalcula.
- Las aplicaciones anuladas permanecen en el historial.
- Las operaciones son atómicas.
- Las restricciones existen también a nivel de base de datos.
- Las pruebas automáticas pasan.

12. Riesgos del bloque

Riesgo	Impacto	Tratamiento
Confundir clasificación con conciliación	Alto	Separar modelos y acciones
Modificar datos originales	Alto	Campos originales de solo lectura
Sobreaplicar movimiento	Alto	Servicio, bloqueo y validación
Sobreaplicar destino	Alto	Control de saldo del destino
Aplicar ingreso a egreso	Alto	Validación por dirección
Dos usuarios aplican simultáneamente	Alto	<code>select_for_update()</code>
<code>clean()</code> no ejecutado	Alto	Servicio y restricción de base
Editar sin historial	Alto	Desactivar y recrear
Consulta lenta	Medio	Anotaciones e índices
Confundir B009 con B010	Alto	Separación bancaria/factura

13. Resultado consolidado

Al finalizar este bloque existirá el siguiente flujo:

Movimiento importado
→ visible en listado
→ revisado en detalle
→ clasificado administrativamente
→ aplicado a una o varias operaciones
→ saldo recalculado
→ estado actualizado
→ historial registrado

La salida de este bloque habilita:

B010 – Un movimiento paga varias facturas
B011 – Conciliación parcial
B013 – Cruce manual asistido
B014 – Cruce con rendiciones y facturas